

Generated Drifts

X and a hand-designed
Randers field ω
`run.swiss_roll.generated.py`
`/ run.mammoth.generated.py:`
`make.*_randers()`

Calculated Drifts

X alone – no field
hand-crafted anywhere

plain directed geodesic (no field):

$D_{asym} =$
`compute.dist_matrix( $X, k,$`
`None, directed=True)`
– asymmetric purely from the
directed k -NN adjacency itself

`randers_bridge.py:`
`compute.dist_matrix()` L32

$\omega =$

`compute.highdim_drift( $X,$`
 `$D_{asym}, k,$  clip_delta)`
– feeds `compute.drift()` the
raw AMBIENT coordinates
 X (not the embedding Y)

`randers_bridge.py:`
`compute.highdim_drift()` L155

`fdgl.pipeline()` – **Locate + Apply**
(shared by both families: hand-crafted
vs. derived ω is the only difference left)

Locate: $X_{aug} = [X; X + \omega]$, plain Euclidean geodesic
 $D_{sym,aug}$ over all $2n$ points (knn/Dijkstra, ω NOT injected here)

`randers_bridge.py: fdgl.pipeline()` L296--302

(real,real) block: $D_{real,asym} = \text{compute.dist_matrix}(X, \omega, \text{return_adjacency}=\text{True})$
(directed Dijkstra, computed ONCE here and cached),
then $D_{sym,aug}[n,:n] = 0.5(D_{real,asym} + D_{real,asym}^T)$

`randers_bridge.py: fdgl.pipeline()` L304--311

`classical.mds( $D_{sym,aug}, d$ )` on all $2n$ points
– one deterministic placement, no training

`randers_bridge.py: fdgl.pipeline()` L313--317

split $Y_{aug0} \rightarrow Y_{real0}, Y_{virtual0}$
 $B_{located} = Y_{virtual0} - Y_{real0}$, clipped, FROZEN
 $Y_{init} = Y_{real0}$

`randers_bridge.py: fdgl.pipeline()` L318--323

Apply: REUSE $D_{asym} = D_{real,asym}$, $bln_{asym} = bln_{real}$ from the locate
step above (identical $X/k/\omega$ /adjacency $\rightarrow$ identical result – no longer re-
computed, a former redundant second Dijkstra call removed 2026-09-17);
 $D_{geo} = \text{compute.dist_matrix}(X, \text{None})$ freshly built (drift-free counterpart)

`randers_bridge.py: fdgl.pipeline()` L338--351

`fdgl.low_dim()` – **Internal Pipeline**
(shared code, runs identically for both families from here on)

build directed k -NN membership: $A, knn_mask = _knn_weights(D_{asym}, k)$

`randers_fdgl.py: L367, \_knn\_weights()` L93

A_{geo} : Euclidean-weight counterpart, only needed if
`randers.attractive/randers.repulsive=False` – $A_{geo} = _knn_weights(D_{geo}, k)$ if
 D_{geo} given, else falls back to A (warns: drift may leak into the “Euclidean” term)

`randers_fdgl.py: L369--383`

$(a, b_{param}) = \text{find_ab_params}(\text{spread}, \text{min_dist})$; $N =$
`\_compute\_N( $D_{asym}$ )` (once, fixed for the whole run)

`randers_fdgl.py: L385--386, find\_ab\_params()` L125, `\_compute\_N()` L185

if `force_model = “fr_gravity”`: $A_{edges} = knn_mask$
 $\vee knn_mask^T$ (undirected union); $k_{nat} = \text{fr.k}$ or $1/\sqrt{n}$

`randers_fdgl.py: L390--395`

$Y = Y_{init_override}$ if given, else `classical.mds( $D_{asym}, d$ )`; $Y_{init} = Y.copy()$

`randers_fdgl.py: L397--401, classical.mds()` L144

$B = \mathbf{0}$ if B_{fixed} is None else $B_{fixed}.copy()$; $M = \text{node_mass}$ or all-ones; if B_{fixed}
given and `scale_B.fixed_by_knn_distance`: precompute its unit direction once

`randers_fdgl.py: L403--413`

snapshot pre-capture (epoch-0 B , if `snapshot_every` given); verbose config print

`randers_fdgl.py: L415--456`

----- EPOCH LOOP: repeats n_epochs times -----

ramp schedule: $s = 0$ (first 30% of epochs) $\rightarrow$ linear 0 $\rightarrow 1$
(30–70%) $\rightarrow s = 1$ (last 30%); $s = 1$ always if `ramp=False`

`randers_fdgl.py: L461--465`

recompute pairwise geometry from the CURRENT Y : `diff,  $d_{mat}$ , unit vectors  $e$`

`randers_fdgl.py: L467--469`

B_{fixed} given $\rightarrow B = s \cdot B_{fixed}$ (frozen direction;
optional live k -NN-distance magnitude rescale)

`randers_fdgl.py: L471--483`

`use_drift=True`, $B_{fixed} = \text{None}$ $\rightarrow B =$
 $s \cdot \text{compute_drift}(N, knn_mask, k, Y, \text{clip_delta})$

`randers_fdgl.py: L484--486, compute\_drift()` L201

build (ρ_r, g_r) Randers-perturbed and (ρ_e, g_e) Euclidean pairs; select
 $\rho_{attr}/g_{attr}, \rho_{rep}/g_{rep}$ per `randers.attractive/randers.repulsive`

`randers_fdgl.py: L488--495`

`force_model=“umap”`: UMAP (a, b) -curve
`attr.cooff/rep.cooff`, weighted by A_{attr}/A_{rep}

`randers_fdgl.py: L499--512`

`force_model=“fr_gravity”`: Bannister et al.: $f_a = \rho/k_{nat}$ (edges only,
 A_{edges}), $f_r = k_{nat}^2/\rho^2$ (all pairs)

`randers_fdgl.py: L513--523`

`step = force.sum(axis=1)` – net force on each y_i

`randers_fdgl.py: L524--529`

OPTIONAL, additive: if `negative_sampling`: draw S fresh random negatives
per node this epoch, replace/augment repulsion (formula differs by `force_model`)

`randers_fdgl.py: L533--558`

OPTIONAL, additive: if `use_gravity`: `step +=  $\gamma M w_{grav} \cdot \text{gravity\_term}$`
(pulls $y_i \rightarrow y_i + b_i$; plain linear or Randers pseudo-edge curve)

`randers_fdgl.py: L560--590`

OPTIONAL, additive: if `use_virtual_neighbor`: `step`
`+= attr.cooff_v \cdot B` (unconditional $(k+1)$ -th neighbour pull)

`randers_fdgl.py: L593--597`

clip `step` to `[-grad_clip, grad_clip]`; decaying lr `cur_lr = lr \cdot (1 -`
`epoch/ $n\_epochs$ )`; $Y += \text{cur_lr} \cdot \text{step}$; snapshot capture if `snapshot_every`

`randers_fdgl.py: L599--606`

----- end of epoch loop, next epoch
(or exit after n_epochs) -----

final B consistency recompute: live drift recomputed once more against the
FINAL Y ; or B_{fixed} magnitude rescaled against the final Y ’s k -NN distance

`randers_fdgl.py: L616--628`

return $\{Y, Y_{init}, B, \mu, \mu_{directed}, knn_mask, N, a, b_{param}, \text{history}\}$

`randers_fdgl.py: L633--641`